Dagster

Description

- Dagster is an open-source orchestration platform that can be used to conduct ETL / ELT workflows.
- It supports several connections and uses a code-first approach to building directed acyclic graphs (DAGS).

Key Concepts

1. `resource` : Anything that Dagster can use to as part of an operation (e.g. database connection, API).
2. `op` : Short for operation. A particular step that needs to be conducted in the pipeline.
3. `job` : The pipeline that strings together all the operations into one.

Setup and Installation

1. Install dagster via `pip` .

```
pip install dagster
```

2. Create a `.env` file and include the URL to PostgreSQL.

```
DAGSTER_POSTGRES_URL="postgresql://postgres:dea2025!@172.93.55.84:8070/data_engineer_ac
```

3. Create a YAML configuration file called `config.yaml`

```
resources:
  postgresql_db:
    config:
      postgres_url:
        env: DAGSTER_POSTGRES_URL
```

Basic Usage - Creating a Pipeline

- This pipeline will enable you to connect to PostgreSQL to get access to a table, perform a transformation, and then export the transformed DataFrame as a `.csv` file.

Add imports

- File name: `pipeline.py`

```
import os
import pandas as pd
from dagster import job, op, resource, Field, StringSource
from sqlalchemy import create_engine
```

Define a Resource

```
...

# Resources are decorators
## Contains a parameter called config_schema
@resource(config_schema={
    "postgres_url" : Field(StringSource, description="PostgreSQL connection")
})
# resource will decorate the below function
def postgresql_db(context):
    return create_engine(context.resource_config["postgres_url"])
```

Setup the Extract Table Operation

```
...

# op is a decorator
## Contains a parameter called required_resource_keys
## Used to enforce prerequisites for the operation
@op(required_resource_keys={"postgresql_db"})
def extract_table(context) -> pd.DataFrame:
    # Get the engine
    engine = context.resources.postgresql_db

    # Get the DataFrame
    df = pd.read_sql_table(table_name="hospital_readmissions_json", schema='dea', con=e

    # Add a log statement
    context.log.info(f"Loaded {len(df)} rows from the table.")

    return df
```

Setup the Transformation Operation

```
...

@op
def transformation(df: pd.DataFrame) -> pd.DataFrame:
    # Get the first row from raw_json column
    row = df.iloc[0]['raw_json']

    # Transform the row into a DataFrame
    transformed_df = pd.json_normalize([row])

    # Perform a transpose
    transformed_df = transformed_df.T

    # Reset the index and drop the extra index column
    transformed_df.reset_index(inplace=True)

    # Rename the columns
    transformed_df.columns = ['tag', 'value']

    return transformed_df
```

Export the Transformed DataFrame to a CSV

```
...

@op
def write_to_csv(df: pd.DataFrame):
    # Define the output path
    output_path = os.getcwd() + "/output.csv"

    # Export the DataFrame to a CSV
    df.to_csv(output_path, index=False)

    print(f"Wrote transformed data to {output_path}.")
```

Define the Job

```
...
# job is a decorator
## resource_defs is a parameter of the decorator
@job(resource_defs={"postgresql_db" : postgresql_db})
def postgres_to_csv_job():
    # Use the internal functions of operations in here
    df = extract_table()

    transformed_df = transformation(df)

    write_to_csv(transformed_df)
```

Use Dagster CLI to Execute the Job

```
dagster job execute -f pipeline.py -c config.yaml
```